# Final Project Report

# on

# Credit Card Fraud Detection Using Machine Learning Models

Team Members: **Sakshi Sanghi & Dharani Manne**

MS in Data Science, Wright State University

CS 7830 – Machine Learning

Dr. Tanvi Banerjee

# Table Of Contents


**1. INTRODUCTION ..... 3**

**2. METHODS ..... 4**

2.1. Data Preprocessing ..... 4
2.2. Models Implemented ..... 4
*2.2.1. Logistic Regression ..... 4*
*2.2.2. Linear SVM ..... 5*
*2.2.3. XGBoost ..... 5*
2.3. Hyperparameter tuning ..... 6

**3. RESULTS ..... 7**

3.1. Evaluation Metrics ..... 7
3.2. Confusion Matrix ..... 7
3.3. Precision Recall Curves ..... 8
3.4. Results Comparison table ..... 9
3.5. ROC Curve Comparison ..... 9

**4. DISCUSSION / ANALYSIS ..... 10**

4.1. Model Performance Comparison ..... 10
4.2. Impact of Class Imbalance Handling ..... 11

**5. LIMITATIONS / TECHNICAL CHALLENGES ..... 12**

**6. CONCLUSION ..... 12**

**7. PROJECT REPOSITORY AND TEAM CONTRIBUTIONS ..... 13**

7.1. GitHub Repository ..... 13
7.2. Team Members ..... 13
7.3. Contribution Breakdown ..... 13

# 1. Introduction

Credit card fraud is one of the most prevalent and costly forms of financial crime in the digital economy. As electronic transactions become increasingly common, fraudsters continuously evolve their tactics to exploit vulnerabilities in payment systems. The ability to detect fraudulent transactions in real time while minimizing false alarms that inconvenience legitimate customers that presents a significant machine learning challenge.

This project addresses the binary classification problem of distinguishing fraudulent credit card transaction (Class =1) from legitimate ones (Class =0). We applied and compared three machine learning algorithms: logistic Regression, Linear Support Vector Machine (SVM) and XGBoost on the dataset. Our objective is to evaluate each model's ability to correctly identify fraud while managing the inherent class imbalance of the dataset.

The data set contains two like 284,807 transactions made by European cardholders in September 2013 of which only 492(approximately 0.17%) are fraudulent. This extreme imbalance makes standard accuracy and insufficient metric and requires careful consideration of evaluation approaches such as precision, recall, F1 score, and the Area Under the ROC Curve (AUC).

The broader motivation for this work lies in these societal and financial impacts of the fraud deduction false negatives (missed frauds) result in monetary losses and customer trust while false positives (incorrectly flagged transactions) create friction and customer dissatisfaction, building robust interpretable models that balance these tradeoffs is therefore of high practical value.

# 2. Methods

## 2.1. Data Preprocessing

The dataset used is the Kaggle Credit Card Fraud Detection dataset, containing 284,807 transactions described by 31 features. The features V1 through V28 are the result of a Principal Component Analysis (PCA) transformation applied to the original transaction data for confidential reasons. The target variable is Class where 1 denotes fraud and 0 denotes a legitimate transaction.

The following preprocessing steps were performed:

- Missing value check: A complete check for null values was performed across all 31 columns. No missing values were found in the dataset, confirming its cleanliness.
- Duplicate Detection: All rows were inspected for duplicates. The dataset contained no duplicate records.
- Class Imbalance Analysis: The target variable exhibited severe class imbalance – 284,315 legitimate transactions (99.83%) versus only 492 fraudulent ones (0.17%). This imbalance was addressed using SMOTE (Synthetic Minority Over-sampling Technique). Withing the XGBoost and Linear SVM pipelines to generate synthetic minority class samples during training.
- Train/Test Split: The dataset was divided into training (80%) and test (20%) sets using stratified sampling with random_state = 42, preserving the class ratio across splits. This resulted in 227,845 training samples and 56,962 test samples, with 394 and 98 fraud cases respectively.
- Feature Scaling: StandardScalar was applied within the SVM and XGBoost pipelines to normalize feature magnitudes, which is particularly important for distance and gradient based algorithms.

## 2.2. Models Implemented

### 2.2.1. Logistic Regression

Logistic Regression is a linear probabilistic classifier well-suited for binary classification. It models the probability that a transaction belongs to fraud using a sigmoid function applied to a linear combination of input features. Three variants were implemented:

- No Regularization (penalty = None, solver =Ibfgs): A baseline logistic regression trained with class_weight =' balanced' to compensate for the class imbalance. This model ran for up to 2,00- iterations to ensure convergence.

- L1 Regularization (Lasso, penalty ='l1', solver=saga): Adds a penalty proportional to the absolute value off model coefficient, encouraging sparsity by driving some coefficients to zero. This can act as a form of automatic feature selection.
- L2 Regularization (Ridge, penalty='l2', solver =Ibffgs): Adds a penalty proportional to the squared value of coefficients, shrinking them toward zero without full elimination. This is the default regularization in sklearn and tends to be more stable.

All logistic regression variants used class_weight ='balanced' to handle the imbalanced target distribution. The ROC curve was computed using the model's decision_function output (raw log-odds scores).

### 2.2.2. Linear SVM

The Support Vector Machine (SVM) with a linear kernel identifies the hyperplane that maximizes the margin between the two classes. The LinearSVC implementation from scikit-learn was used, which is optimized for large linear problems. The model was embedded in an imbalanced-learn Pipeline with the following component:

- StandardScaler: Normalizes all 30 features to zero mean and unit variance.
- SMOTE: Generates synthetic fraud samples in the training set to address class imbalance, creating a balanced distribution before the classifier is trained.
- LinearSVC (C=1.0, class_weight ='balanced'): Trains the linear decision boundary with a regularization parameter C=1.0. The balanced class weight further adjusts for any residual imbalance.

ROC scores were computed using the model's decision_function which return signed distance from the decision hyperplane and serves as a confidence score for classification.

### 2.2.3. XGBoost

Extreme Gradient Boosting (XGBoost) is an ensemble learning method that builds an addictive sequence of decision trees, with each successive tree correcting the errors of its predecessors. It is widely recognized for its high predictive accuracy on tabular data. The implementation used the following configuration:

- Pipeline: StandardScaler followed by SMOTE for oversampling, then the XGBClassifier.
- n_estimators = 200: The ensemble consists of 200 boosted trees.
- max_depth = 4: Limits tree depth to avoid overfitting.
- learning_rate =0.1: controls the contribution of each tree(shrinkage).
- Subsample =0.8, colsample_bytree =0.8: Row and column subsampling for regularization.
- Objective='binary: logistic:' Output probabilities for binary classification.

XGBoost is particularly well-suited for imbalanced datasets due to its native support for sampling weighting and its ability to focus learning on hard-to-classify examples through the boosting mechanism.

## 2.3. Hyperparameter tuning

Hyperparameter tuning was performed to improve model performance. The tuning was mainly focused on parameters that directly affect model learning and convergence.

For Logistic Regression the following parameters were adjusted:

- Learning Rate
- Number of Iterations
- Regularization

For Linear SVM, minimal tuning was performed. The model was primarily evaluated using default parameters, focusing on comparing its performance with other models.

Hyperparameter tuning was primarily performed for the XGBoost model via threshold optimization. Rather than relying on the default 0.5 classification threshold, a sweep over thresholds from 0.01 to 0.99 was conducted to identify the threshold that maximized the F1-score for the fraud class (Class =1).

The optimal threshold was identified as 0.99, yielding the best F1-score of 0.8427 for the fraud class with a precision of 0.9375 and recall of 0.7653. This represents a substantial improvement over the default threshold, demonstrating the importance of threshold selection in imbalanced classification problems.

# 3. Results

## 3.1. Evaluation Metrics

Given the extreme class imbalance, accuracy alone is an inadequate measure of model performance which is a naive classifier that labels all transactions as legitimate would achieve 99.83% accuracy while detecting zero fraud cases. The following metrics were therefore used for comprehensive evaluation:

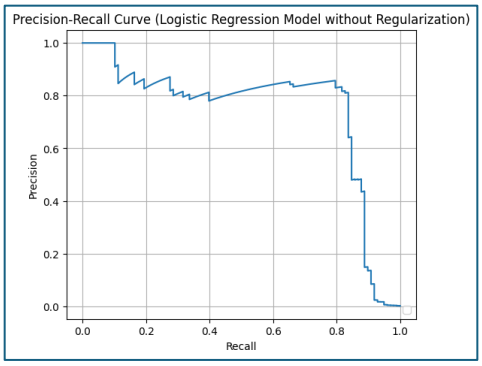
- Accuracy: Overall correctness of predictions
- Precision: Measures how many predicted fraud cases are fraud
- Recall: Measures how many actual fraud cases were correctly detected
- F1 Score: Harmonic mean of precision and recall
- AUC-ROC: Measures model performance across all classification thresholds

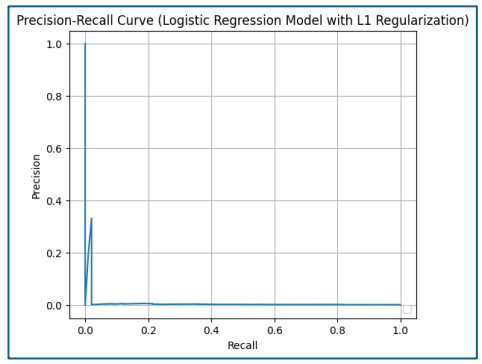
## 3.2. Confusion Matrix

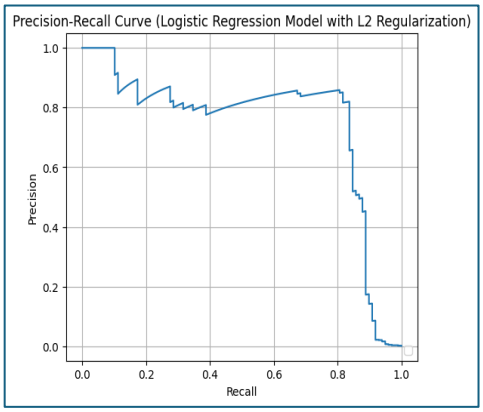
Confusion matrices were generated for each model and visualized using ConfusionMatrixDisplay from scikit-learn. Below is a summary of the key classification outcomes on the held-out test set (56,962 transactions, 98 fraudulent):

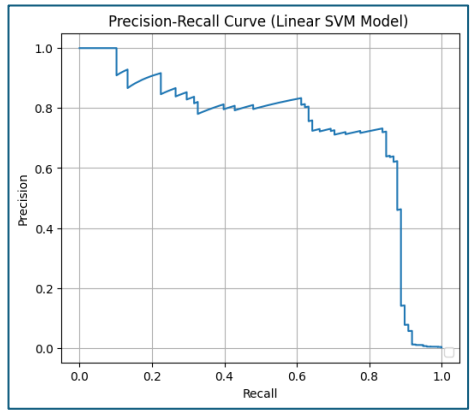
| Model | True Positives (Fraud Caught) | False Negatives (Fraud Missed) | False Positives (False Alarms) | True Negatives (Correct Legit) |
|---|---|---|---|---|
| **LR (No Regularization)** | 90 | 8 | 1455 | 55409 |
| **LR (L1 Regularization)** | 2 | 96 | 18 | 56846 |
| **LR (L2 Regularization)** | 90 | 8 | 1475 | 55389 |
| **Linear SVM** | 89 | 9 | ~ 1271 | ~ 55593 |
| **XGBoost (default)** | 87 | 11 | ~ 200 | ~ 56664 |
| **XGBoost (threshold=0.99)** | 75 | 23 | 5 | 56859 |

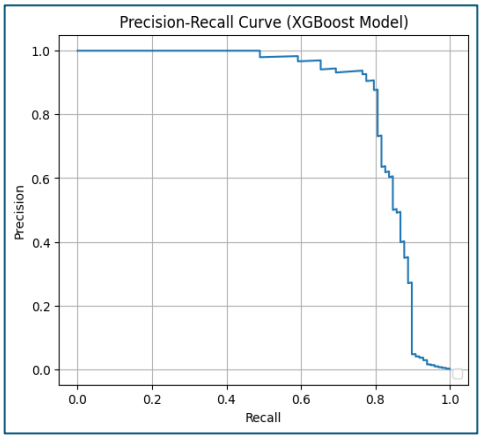
## 3.3. Precision Recall Curves

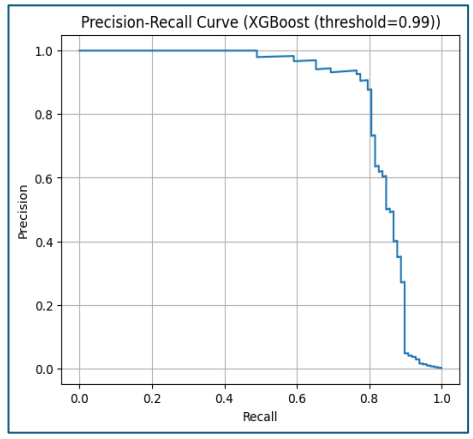


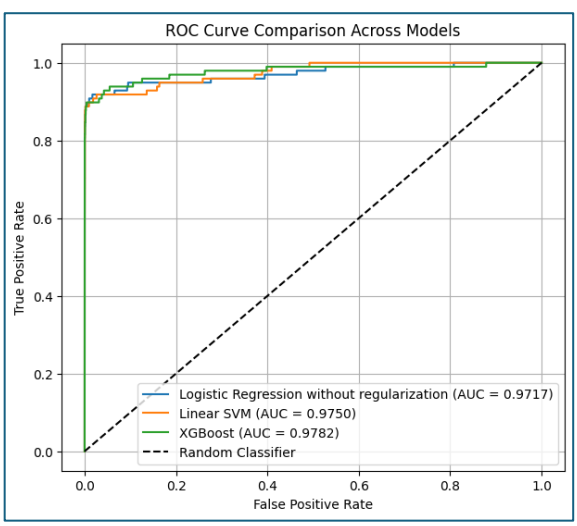
## 3.4. Results Comparison table

The table below presents the complete evaluation metrics for each model configuration tested. For XGBoost results are shown at both the default threshold (0.5) and the optimized threshold (0.99).

| Model | Accuracy | Precision (Class 0) | Precision (Class 1) | Recall (Class 0) | Recall (Class 1) | F1 (Class 0) | F1 (Class 1) | AUC |
|---|---|---|---|---|---|---|---|---|
| **LR (No Reg.)** | 97.45% | **0.9999** | 0.0586 | 0.9746 | 0.9184 | 0.9871 | 0.1102 | 0.9717 |
| **LR (L1 Reg.)** | 99.80% | 0.9983 | 0.1000 | 0.9997 | 0.0204 | 0.9990 | 0.0339 | 0.6359 |
| **LR (L2 Reg.)** | 97.42% | 0.9999 | 0.0580 | 0.9743 | **0.9184** | 0.9869 | 0.1092 | 0.9723 |
| **Linear SVM** | 97.73% | 1.0000 | 0.0600 | 0.9778 | 0.9082 | 0.9888 | 0.1120 | 0.9750 |
| **XGBoost (default)** | 99.63% | 0.9998 | 0.3031 | 0.9965 | 0.8878 | 0.9981 | 0.4519 | **0.9782** |
| **XGBoost (tuned)** | **99.95%** | 0.9996 | **0.9375** | **0.9999** | 0.7653 | **0.9998** | **0.8427** | **0.9782** |

## 3.5. ROC Curve Comparison

The ROC curve comparison was implemented in the ROC-AUC_modelComparison notebook. Custom roc_curve and auc_manual functions were implemented using the trapezoidal rule for AUC calculation, and the results were compared across the three primary models:



| Model | AUC Score | Ranking |
|---|---|---|
| **Logistic Regression (No Regularization)** | 0.9717 | 3rd |
| **Linear SVM** | 0.9750 | 2nd |
| **XGBoost (Tuned)** | **0.9782** | **1st** |

All three models achieved strong AUC scores above 0.97, indicating excellent discriminative ability across classification thresholds. XGBoost consistently outperformed the others. The ROC curves were plotted together alongside a random classifier baseline (AUC =0.5), demonstrating the significant margin of improvement all models achieve over random chance.

# 4. Discussion / Analysis

## 4.1. Model Performance Comparison

- All models achieved strong performance with AUC > 0.97, indicating excellent ability to distinguish between fraud and non-fraud transactions.
- **XGBoost** (Tuned) performed the best with highest AUC (~0.9782) and best overall balance between precision and recall
- **Linear SVM** showed stable performance with an AUC ~0.9750. This model is better than Logistic Regression but slightly below XGBoost.
- **Logistic Regression (No Regularization)** with an AUC ~0.9717 is good baseline model but limited in capturing complex patterns
- Some models achieved very high accuracy (~ 99%), but this is due to class imbalance. For example: LR (L1 Regularization) with an Accuracy = 99.80% and Recall = 0.0204 (very poor). Hence, this suggests that high accuracy doesn't means good fraud detection and the model is biased towards majority classes.

### Effect of Regularization

- *L1 Regularization* has increased accuracy performance but drastically reduced recall resulting in poor fraud detection.
- *L2 Regularization* is like non-regularized model but with slight improvement in stability.

### Precision vs Recall Trade-off

- *Logistic Regression (No Regularization)* with high Recall (~0.9184) suggests it detects most fraud and very low Precision (~0.0586) suggests it has many false alarms

- *XGBoost (Default)* results in better balance with Precision (~0.3031) and Recall (~0.8878)
- *XGBoost (Threshold = 0.99)* has very high Precision (~0.9375) and Lower Recall (~0.7653)
- Above performance metrices suggests that by increasing threshold results in fewer false positives but more missed fraud cases

### Confusion Matrix Analysis

- *Logistic Regression* model has high FP which means that it detects fraud but causes many false alerts.
- *XGBoost (Default)* shows balanced behavior with Low FP (~200) and Low FN (~11)
- *XGBoost (High Threshold)* has very low FP (~5) and Higher FN (~23)
- It suggests that lower threshold results in better fraud detection and Higher threshold will give fewer false alarms

### Final Comparison Summary

- **Best Overall Model**: XGBoost (Tuned)
- **Best Recall**: Logistic Regression (No Regularization)
- **Best Precision**: XGBoost (High Threshold)
- **Best Balance**: XGBoost (Default)

## 4.2. Impact of Class Imbalance Handling

The use of SMOTE in the XGBoost and SVM pipelines is a key design decision. SMOTE generates synthetic minority class samples by interpolating between existing fraud examples in feature space, increasing the fraud representation int raining from 0.17% to a balanced 50/50 split. This allows both models to learn discriminative boundaries for the fraud class without simply memorizing real fraud cases.

Logistic regression models instead relied on class_weight='balanced', which adjust sample weights inversely proportional to class frequency. Both approaches address imbalance, but SMOTE's augmentation of the training data tends to yield richer learned representations, which likely contributes to XGBoost's superior performance.

# 5. Limitations / Technical Challenges

- Anonymized Features: PCA-transformed features (V1–V28) limit interpretability and prevent domain-specific feature engineering.
- Temporal Considerations: The dataset is not split chronologically, which may introduce minor temporal leakage. In real-world systems, time-based validation would be more appropriate.
- Static Dataset: The data (from 2013) may not reflect evolving fraud patterns, limiting the model's ability to generalize to current scenarios.
- Severe Class Imbalance: With only 0.17% fraud cases, standard metrics like accuracy are misleading, requiring techniques like SMOTE, class weighting, and threshold tuning.
- Model-Specific Limitations:
    - Logistic Regression (L1) underperformed due to strong regularization.
    - SVM training becomes computationally expensive with large, SMOTE-augmented data.
    - Non-linear models were not explored due to constraints.
- Limited Hyperparameter Tuning: Only XGBoost received detailed tuning; other models could improve with systematic grid search or optimization techniques.
- Technical Constraints:
    - XGBoost required correct package installation (xgboost).
    - Custom ROC/AUC implementation required careful handling to ensure correctness.

# 6. Conclusion

This project implemented and evaluated three machine learning models Logistic Regression (with multiple regularization configurations), Linear SVM and XGBoost for credit card fraud detection on a highly imbalanced real-world dataset.

The key findings are as follows:

- *XGBoost with threshold tuning* emerged as the best-performing model, achieving an F1-score of 0.8427, precision of 0.9375, and AUC of 0.9782. The identification of an optimal threshold (0.99) significantly improved precision while maintaining acceptable recall, demonstrating the importance of decision threshold optimization in imbalanced classification problems.
- *Linear SVM* achieved a competitive AUC of 0.9750 and strong recall (~91%) when combined with SMOTE, making it suitable for scenarios were detecting as many fraud cases as possible is critical.

- *Logistic Regression (No Reg / L2)* provided a stable and interpretable baseline (AUC ~0.97) but suffered from very low precision (~6%), resulting in a high number of false positives. L1 regularization significantly degraded performance, likely due to excessive penalization.
- SMOTE-based oversampling proved more effective than class weighting alone, particularly for XGBoost, by improving the model's ability to learn minority class patterns.
- The AUC metric, computed using a custom implementation, confirmed consistent model ranking:
  **XGBoost > SVM > Logistic Regression (L2) > Logistic Regression (L1).**

From a practical perspective, **XGBoost with threshold tuning is the most suitable model for deployment** as it achieves high precision (93.75%) reducing false alarms and operational costs, while still maintaining reasonable recall (76.53%) to detect most fraud cases.

**Overall, this study highlights that in highly imbalanced datasets model performance is not solely dependent on the algorithm choice but significantly influenced by data balancing techniques and threshold optimization strategies.**

# 7. Project Repository and Team Contributions

## 7.1. GitHub Repository

The complete project implementation including all notebooks, code and report is available at the following GitHub repository:
GitHub Link: https://github.com/sakshisanghi03/ML_CS7830_FinalProject_SD

## 7.2. Team Members

Member 1: Sakshi Sanghi
Member 2: Dharani Manne

## 7.3. Contribution Breakdown

| Contributor | Contribution |
|---|---|
| **Dharani** | • Performed EDA and data preprocessing<br>• Implemented Linear SVM model |
| **Sakshi** | • Implemented XGBoost model and<br>• Logistic Regression models (no reg/L1 reg/L2 reg) |
| **Sakshi & Dharani** | • Conducted model evaluation and performance comparison<br>• Worked on ROC-AUC analysis and threshold tuning<br>• Collaborated on report writing and final submission |